# Feasibility Study and Implementation Plan

## Travel Itinerary and Expense Organizer (MySQL Workbench implementation)

Prepared by: Team (add names) | Date: 23 October 2026 | Database: `travel_itinerary_db` (version 2)

## 1. Project summary

The project turns our approved ER diagram into a working relational database. The system lets travellers plan trips day by day, track bookings and expenses, split costs among group members, and reserve hotel rooms with payments and reviews.

| Item | Value |
|---|---|
| Entities (tables) | 21 (users, destinations, places, trips, itinerary days/items, bookings, expenses, reviews, hotel module, and lookup tables) |
| Relationships (foreign keys) | 27, each with an explicit ON DELETE / ON UPDATE rule |
| Special modelling | IS-A (hotels is a subtype of places), 2 weak entities (hotel_payments, hotel_reviews), composite foreign key (rooms to room_types) |
| Business rules in the database | 12 triggers (6 rules, each on insert and update), 1 view, 2 stored procedures, 30+ CHECK / UNIQUE constraints |
| Sample data | 312 rows across all 21 tables, realistic Indian travel scenario in INR |
| Deliverables | `01_travel_itinerary_v2_full.sql` (schema, triggers, procedures, view, data) and `02_demo_queries.sql` (12 queries, 14 rule tests, procedure demo) |

### Objectives

1. Create every table exactly as drawn in the ER diagram, with the same keys, constraints and referential actions.
2. Load meaningful sample data that exercises every relationship.
3. Prove the design works: queries return correct answers and invalid data is rejected by the database itself.
4. Document the technology choice, plan, team split and risks (this document).

## 2. Feasibility analysis

### 2.1 Technical feasibility: feasible

- Every feature of the diagram has a native MySQL equivalent: composite primary and foreign keys, ENUM, DECIMAL money types, CHECK constraints (enforced from MySQL 8.0.16), BEFORE triggers with SIGNAL for business rules, views for derived values, and transactions with `SELECT ... FOR UPDATE` for safe booking.
- Derived attributes (nights, total cost, amount paid, balance due, actual spend, remaining budget) are not stored. They are computed in the view or in queries, which keeps the schema normalised.
- Rules that SQL constraints cannot express (no overlapping activities in a day, no double-booked room, only completed stays can be reviewed, expense must belong to the same trip) are implemented as triggers.
- The one rule that cannot be enforced declaratively, that a trip owner is also a member with role owner, is handled by the `sp_create_trip` transaction and checked by an integrity query (Q12).
- Risk level: low. All techniques are covered in a standard DBMS course.

### 2.2 Operational feasibility: feasible

The team only needs a laptop with MySQL Server and MySQL Workbench. Setup takes under 30 minutes. The scripts are re-runnable (they drop and rebuild the database), so anyone can restore a clean state before a demo.

### 2.3 Economic feasibility: feasible (cost: Rs 0)

MySQL Community Edition, MySQL Workbench and (optionally) Python with Streamlit are free and open source. No paid cloud service is needed.

## 2.4 Schedule feasibility: feasible

The core database is already scripted. Remaining work is installation, execution on the team's own machines, screenshots, EER diagram export and report writing. A 10-day plan is given in section 7; the critical path is about 4 days.

## 2.5 Data and legal feasibility: feasible

- All sample data is synthetic: fictional people, fictional hotels, `.example` e-mail domains and fake phone numbers. No real personal data is used.
- Passwords in the demo are stored as SHA2 hashes only to avoid plain text. A real application must use bcrypt or argon2 with a salt; this is stated as a limitation.
- Card numbers are never stored; only a transaction reference is kept.

## 2.6 Risk register

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Server and Workbench version mismatch | Medium | Medium | Use MySQL 8.4 LTS; run the scripts in Workbench; check connection before the demo (see 3.1) |
| CHECK constraints ignored on very old MySQL | Low | High | Require 8.0.16 or newer; the quick-check query at the end of script 01 confirms the object counts |
| Trigger creation blocked by privileges or binary logging | Low | Medium | Use the `root` account locally; if error 1419 appears, run `SET GLOBAL log_bin_trust_function_creators = 1;` |
| Workbench stops a script at the first error | Medium | Low | Section C of the demo file is run statement by statement and wrapped in a transaction that is rolled back |
| A teammate edits the schema separately and versions diverge | Medium | Medium | One shared GitHub repository; only the schema owner merges changes to script 01 |
| Data entry error in sample data | Low | Low | Sample data was dry-run against all constraints and trigger rules before delivery (section 8) |

**Overall verdict: the project is technically, operationally, economically and schedule-wise feasible. Recommendation: proceed.**

# 3. Technology stack

## 3.1 Core stack (required)

| Layer | Choice | Why |
|---|---|---|
| Database server | MySQL Community Server 8.4 LTS | MySQL 8.0 reached end of life in April 2026, so 8.4 LTS is the supported long-term release. The scripts also work on 8.0.16 or newer. |
| Client and modelling | MySQL Workbench | Runs SQL scripts, shows result grids, and generates the EER diagram. The new Workbench 26.x line was announced in September 2026; its announcement did not list the EER diagram editor, so check your installed version. If the EER editor is missing, install classic Workbench 8.0.x for diagramming only. |
| Language | SQL (MySQL dialect) | DDL (CREATE), DML (INSERT, UPDATE, DELETE), DQL (SELECT), TCL (START TRANSACTION, COMMIT, ROLLBACK) and stored program language (triggers, procedures) |
| Storage engine | InnoDB | Required for foreign keys, transactions and row locking |
| Character set | utf8mb4 | Full Unicode, safe for names and symbols |
| Version control | Git and GitHub | Shared repository, one commit per task |

## 3.2 Optional demo layer (only if time permits)

A small web front end makes the demo more impressive but is not needed for the Workbench submission.

| Package | Purpose |
|---|---|
| Python 3.11 or newer | Application language |
| `streamlit` | Quick web UI: trip dashboard, itinerary table, expense chart |
| `mysql-connector-python` | Official driver to run queries and stored procedures |
| `pandas` | Turn query results into tables and summaries |
| `plotly` | Budget-vs-actual and category spend charts |
| `python-dotenv` | Keep database credentials out of the code |

Install with: `pip install streamlit mysql-connector-python pandas plotly python-dotenv`. Keep the app read-mostly: it calls the same queries and `sp_book_room` / `sp_create_trip`, so all business rules stay inside the database.

## 3.3 Design decisions worth explaining in the viva

| Decision | Reason |
|---|---|
| DECIMAL(10,2) for money | Exact arithmetic; FLOAT would cause rounding errors |
| ENUM for statuses and roles | Restricts values to a fixed list, cheap to store |
| `rate_per_night` stored on the reservation | A price snapshot: later changes to the room type's base rate must not change past bookings |
| `hotels.hotel_id` is both primary key and foreign key to `places` | Implements IS-A with a shared key |
| `hotel_payments` key is (reservation_id, payment_no) | Weak entity: a payment has no meaning without its reservation |
| `rooms` has a composite FK (room_type_id, hotel_id) | Guarantees a room's type belongs to the same hotel |

| Decision | Reason |
|---|---|
| Triggers instead of application code | The rule holds no matter which tool inserts the data |
| ON DELETE CASCADE for owned children, RESTRICT for shared master data, SET NULL for optional links | Deleting a trip removes its days and items, but a user who owns trips or a category in use cannot be deleted |

# 4. Data design

## 4.1 Tables by module

| Module | Tables |
|---|---|
| People | users |
| Catalogue | destinations, places, activity_categories, expense_categories, amenities |
| Trip planning | trips, trip_members, itinerary_days, itinerary_items, bookings |
| Money | expenses |
| Community | reviews, saved_destinations |
| Hotel module | hotels, hotel_amenities, room_types, rooms, hotel_reservations, hotel_payments, hotel_reviews |

## 4.2 Normalisation

All tables are in third normal form: every non-key attribute depends on the key, the whole key and nothing but the key. Lookup values (categories, amenities) live in their own tables, many-to-many links use junction tables (trip_members, hotel_amenities, reviews, saved_destinations), and computed values are never stored.

### 4.3 Where each rule is enforced

| Mechanism | Examples |
|---|---|
| PRIMARY KEY / UNIQUE | unique e-mail; one booking per item; one day per date in a trip; no duplicate room number in a hotel; unique confirmation code |
| FOREIGN KEY | 27 links with CASCADE, RESTRICT or SET NULL |
| CHECK | budget at least 0; end date not before start date; rating 1 to 5; payment amount above 0; check-out after check-in |
| Triggers | no overlapping timed items in a day; day_date equals start_date plus day_number minus 1; expense item belongs to the same trip; hotel row must point at a Hotel place; reservation checks (capacity, maintenance, trip dates, booker role, double booking); reviews only for completed stays |
| Transactions | `sp_create_trip` (trip, owner membership and daily rows together); `sp_book_room` (row lock, insert, confirmation code) |
| Integrity query | Q12 confirms every trip owner is a member with role owner |

## 5. Sample data

| Table | Rows | Table | Rows |
|---|---|---|---|
| users | 6 | hotels | 8 |
| destinations | 7 | hotel_amenities | 34 |
| places | 24 | room_types | 16 |
| activity_categories | 7 | rooms | 32 |
| expense_categories | 6 | hotel_reservations | 8 |
| amenities | 8 | hotel_payments | 10 |

| Table | Rows | Table | Rows |
|---|---|---|---|
| trips | 5 | hotel_reviews | 2 |
| trip_members | 15 | reviews | 8 |
| itinerary_days | 24 | saved_destinations | 7 |
| itinerary_items | 47 | bookings | 11 |
| expenses | 27 | **Total** | **312** |

## Trips in the data set

| Trip | Dates | Status | Budget (Rs) | Purpose in the demo |
|---|---|---|---|---|
| Goa Beach Escape | 26 to 30 Dec 2026 | planned | 60,000 | Upcoming trip, partial payments, a failed payment, a cancelled reservation |
| Rajasthan Heritage Tour | 10 to 14 Nov 2026 | planned | 85,000 | Two destinations (Jaipur, Udaipur), two hotel reservations, pending bookings |
| Munnar Monsoon Getaway | 14 to 17 Aug 2026 | completed | 40,000 | Finished trip: settlement query and hotel review |
| Hampi and Coorg Road Trip | 19 to 23 Oct 2026 | ongoing | 35,000 | Trip in progress: completed and checked-in stays |
| Manali Winter Break | 8 to 12 Jan 2027 | planned | 50,000 | Cascade-delete demonstration |

Other deliberate edge cases: one room under maintenance, one skipped itinerary item, items with no place (SET NULL target), expenses with and without an itinerary link, payments by card, UPI, net banking and cash, and a refunded payment.

# 6. Queries and expected results

The demo file contains 12 queries (Section B), 14 rule tests (Section C) and a procedure demo (Section D).

| No. | Query | SQL concepts |
|---|---|---|
| Q1 | Day-wise itinerary of a trip | 4-table JOIN, LEFT JOIN, TIME_FORMAT, ORDER BY |
| Q2 | Budget vs actual spend per trip | LEFT JOIN, SUM, GROUP BY, COALESCE, NULLIF |
| Q3 | Spend by expense category per trip | GROUP BY on two columns |
| Q4 | Settlement: who owes whom | Derived tables, CROSS JOIN, arithmetic |
| Q5 | Reservation summary | View with nights, total, paid and balance |
| Q6 | Free rooms for a date range | NOT EXISTS anti-join, user variables |
| Q7 | Hotel performance | Aggregation over a view, LEFT JOIN |
| Q8 | Best-rated destinations | GROUP BY, HAVING, AVG |
| Q9 | Running total of spend | Window function SUM() OVER |
| Q10 | Trip members and roles | GROUP_CONCAT |
| Q11 | Transport or adventure items without booking | LEFT JOIN ... IS NULL |
| Q12 | Owner-is-member integrity check | Outer join as a data audit |

## Expected output: Q2 (budget vs actual)

| Trip | Budget | Actual spend | Remaining |
|---|---|---|---|
| Goa Beach Escape | 60,000 | 30,000 | 30,000 |
| Rajasthan Heritage Tour | 85,000 | 14,600 | 70,400 |
| Munnar Monsoon Getaway | 40,000 | 30,600 | 9,400 |
| Hampi and Coorg Road Trip | 35,000 | 22,100 | 12,900 |

| Trip | Budget | Actual spend | Remaining |
|---|---|---|---|
| Manali Winter Break | 50,000 | 5,500 | 44,500 |

## Expected output: Q4 (settlement, Munnar trip, total 30,600 split three ways = 10,200 each)

| Member | Paid | Fair share | Net balance |
|---|---|---|---|
| Sneha Iyer | 21,900 | 10,200 | +11,700 (gets back) |
| Karthik Reddy | 6,000 | 10,200 | -4,200 (owes) |
| Ananya Das | 2,700 | 10,200 | -7,500 (owes) |

## Expected output: Q5 (view `v_reservation_summary`)

| Res. | Status | Nights | Total | Paid | Balance |
|---|---|---|---|---|---|
| 1 Goa, Seaside Palms | confirmed | 4 | 26,000 | 18,000 | 8,000 |
| 2 Jaipur | confirmed | 2 | 9,600 | 9,600 | 0 |
| 3 Udaipur | pending | 2 | 13,000 | 2,000 | 11,000 |
| 4 Munnar | completed | 3 | 13,500 | 13,500 | 0 |
| 5 Hampi | completed | 2 | 6,400 | 6,400 | 0 |
| 6 Coorg | checked_in | 2 | 7,600 | 4,000 | 3,600 |
| 7 Manali | pending | 4 | 12,000 | 3,000 | 9,000 |
| 8 Goa, Casa Azul | cancelled | 4 | 19,200 | 0 (refunded) | 0 |

Other expected results: Q6 returns 6 free rooms (Seaside Palms 101 and 202; Casa Azul 101, 102, 201, 202); Q8 shows Goa, Munnar and Hampi tied at 4.5; Q11 returns 6 unbooked transport items; Q12 returns 0 rows.

## Rule tests (Section C of the demo file)

Each of these must be rejected: overlapping activities, a day outside the trip, a double-booked room, too many guests, a room under maintenance, a viewer trying to book, a stay outside the trip dates, a review of an unfinished stay, an expense linked to another trip's item, a hotel row for a non-hotel place, rating 6, negative amount, duplicate e-mail, deleting

a user who owns trips, and deleting a category in use. Two further tests succeed on purpose and are rolled back: ON DELETE CASCADE (deleting a trip removes its 5 days, 7 items, 2 expenses and 1 reservation) and ON DELETE SET NULL (deleting a place keeps the itinerary item).

## 7. Implementation plan and work distribution

The plan assumes a team of four. Rename the members and adjust the split if the team size differs.

### 7.1 Roles

| Member | Role | Responsibilities | Deliverables |
|---|---|---|---|
| Member A | Database architect and team lead | Install MySQL and Workbench, run the schema, own script 01, generate and clean the EER diagram, merge changes | Working database, EER diagram image, final schema review |
| Member B | Data engineer | Review and extend sample data, verify every foreign key, write the data dictionary (table, column, type, constraint, meaning) | Data dictionary, row-count screenshot, data verification notes |
| Member C | Logic developer | Own triggers, procedures and the view; write and run the 14 rule tests; explain transactions and locking | Trigger documentation, screenshots of each rejected statement, procedure demo |
| Member D | Query and presentation lead | Own the 12 queries, add 3 queries of the team's own, prepare screenshots, slides and the viva demo script; build the optional Streamlit page | Query report with outputs, slide deck, demo script, optional app |

Everyone must be able to explain the whole ER diagram; each member presents the part they own.

## 7.2 Ten-day schedule

| Day | Task | Owner |
|---|---|---|
| 1 | Install MySQL 8.4 LTS and Workbench; create the GitHub repository; run script 01 and confirm 21 / 27 / 12 / 1 in the quick check | A (all verify) |
| 2 | Compare the generated tables with the ER diagram, column by column | A, B |
| 3 | Data dictionary; check row counts per table | B |
| 4 | Walk through every trigger; run rule tests C1 to C14 and capture screenshots | C |
| 5 | Run Q1 to Q12; capture results; match them with the expected output in section 6 | D |
| 6 | Generate the EER diagram (reverse engineer from the script or the live database) and tidy the layout | A |
| 7 | Procedure demo (D1 to D5) and view demo; record a short screen capture as a backup | C, D |
| 8 | Optional: Streamlit dashboard; team writes 3 extra queries | D, all |
| 9 | Compile the report and slides; cross-review each other's sections | All |
| 10 | Full rehearsal on a clean machine: run both scripts from scratch | All |

## 7.3 How to run it

1. Open MySQL Workbench and connect to the local server.
2. File, Open SQL Script, choose `01_travel_itinerary_v2_full.sql`, click the lightning bolt (Execute). The last result grid must show tables 21, foreign keys 27, triggers 12, views 1.
3. Open `02_demo_queries.sql`. Run Sections A and B with Ctrl+Enter on each statement. Run Section C one statement at a time, expecting each error. Run Section D in order.

4. For the EER diagram: Database menu, Reverse Engineer, select `travel_itinerary_db`, then File, Export as PNG or PDF. Workbench draws the IS-A subtype as a one-to-one link; label it as IS-A on the exported image.
5. Refresh the schema panel (the refresh icon beside SCHEMAS) if new objects do not appear.

## 8. Verification status and limitations

- Before delivery, the sample data and the trigger rules were simulated in a separate script. It confirmed no duplicate keys, no broken foreign keys, no overlapping activities or room bookings, correct day dates, correct capacities and booker roles, and reviews only on completed stays.
- The SQL itself was written for MySQL 8.0.16 or newer and 8.4 but could not be executed on a live MySQL server while preparing it. The team must therefore perform the first run on their own machine (Day 1) and fix any syntax message that appears. The counts in the quick check are the acceptance test.
- Limitations: the model has no table for user-uploaded photos or multi-currency support; passwords use a demo hash; the settlement query assumes an equal split among all members including viewers.

## 9. Possible extensions

1. Indexes tuned from `EXPLAIN` output on the three heaviest queries.
2. Role-based MySQL accounts (read-only analyst, trip-editor) using GRANT.
3. A scheduled event that marks trips as ongoing or completed from their dates.
4. Seasonal room pricing and a currency column.
5. Streamlit or Flask front end calling the existing procedures.

## 10. Conclusion

The Travel Itinerary and Expense Organizer is fully feasible with free tools and a four-person team. The schema, business rules and sample data are ready; the remaining effort is installation, execution, evidence collection (screenshots, EER diagram) and presentation. We recommend proceeding with MySQL 8.4 LTS and MySQL Workbench as the core stack, and treating the Python and Streamlit layer as an optional enhancement.